Rumi Protocol — Comprehensive Security Review (Internal)

Rumi Protocol — Comprehensive Security Review

Audit anchor commit: `28e9896` (2026-04-22) **Review window:** 2026-04-22 → 2026-05-02 **Status at close:** All findings resolved, deployed to mainnet, regression-fenced. Next review scheduled for early Q3.

Executive summary

Rumi Protocol underwent a structured three-pass internal security review of the entire v2 stack — the CDP backend, stability pool, treasury, 3pool AMM, vault-management AMM, liquidation bot, analytics canister, the icUSD ledger and indexer, and the SvelteKit frontend.

Pass	When	Output	Status
First — discovery	2026-04-22	73 findings across 11 specialist analysis passes	All confirmed real, all triaged
Second — remediation verification	2026-04-23 → 2026-05-01	13 deployment waves shipped; second-pass verification confirmed each fix	60 findings resolved on-chain; rest deferred-by-design or housekeeping
Third — drift + follow-up	2026-05-01 → 2026-05-02	7 follow-up waves; drift check on prior fixes; orphan hunt	All remaining audit items closed; no drift on prior fixes

Net result: every finding from the original review is now in one of three states — *resolved on mainnet with a regression test*, *deferred-by-design until SNS migration*, or *accepted housekeeping*. Eight HIGH-severity findings, all eight fixed. Thirty-six MEDIUM findings, all addressed. Twenty-five LOW findings, addressed or formally accepted.

The protocol has been continuously running on mainnet throughout the review and remediation cycle. No fund losses, no protocol pauses, no emergency interventions.

About the methodology

This was an internal review rather than an external paid engagement. It was structured deliberately to mimic a third-party engagement:

- **Anchored to a single commit** (`28e9896`) so every finding referenced a fixed state of the code, with file and line precision.
- **Eleven specialist analysis passes**, each scoped to one threat model (async-state races, oracle integrity, ICRC transfer hygiene, stable-memory upgrade safety, stability-pool accounting, redemption peg-defense, caller authorization, debt-and-interest accounting, liquidation mechanics, inter-canister-call failure modes, cycle/DoS exposure).
- **Findings recorded as structured JSON** with severity, mechanism, exploit scenario where applicable, and a recommended remediation. Severities calibrated to OWASP-style impact × likelihood.
- **Two-stage verification**: a “dry run” of likely findings before the deep dive, followed by the primary review pass. Anything that survived both was real.
- **Three independent verification passes** after the fixes shipped, looking for incomplete fixes, regressions, and silent drift.

The review covered ~63,000 lines of Rust across 9 canisters, ~30,000 lines of TypeScript/Svelte for the two frontends, the candid surface of every canister, and the integration tests.

First pass — discovery (2026-04-22)

Findings by severity

- **HIGH (8):** issues with direct fund-loss potential or systemic protocol consequences.
- **MEDIUM (36):** correctness, accounting, or DoS issues that don’t directly drain funds but degrade safety, fairness, or operational posture.
- **LOW (25):** hygiene, defense-in-depth, or operational items.
- **INFORMATIONAL (4):** observations, not issues.

High-severity findings — categories

Category	Findings	What was at stake
Caller authorization	2	Inter-canister boundary trust between protocol and stability pool; presence of developer-only `dev_*` endpoints in mainnet wasm
Inter-canister call failure	2	Stranded funds when a multi-step liquidation refund path's second call failed; pending-transfer retries that didn't distinguish ledger-fee changes from transient errors
ICRC transfer hygiene	2	Liquidation collateral and treasury withdrawal flows lacked `created_at_time` deduplication, opening a double-spend window on transient ledger reject
Liquidation mechanics	1	Concurrent partial-liquidations on the same vault could overwrite the first liquidator's payout entry
Stability pool accounting	1	A latent double-deduction in the pool's liquidation absorption path (proportional pre-deduct followed by another proportional deduct), draining depositor balances over many liquidations

How findings were triaged

Each finding got a wave assignment based on:

1. **Independence** — fixes that touched the same files were grouped to avoid merge conflicts.
2. **Severity** — HIGH first, then DoS hardening, then housekeeping.
3. **Test feasibility** — every fix needed either a unit test, a PocketIC integration test ("audit POC"), or both.
4. **Deploy risk** — backend changes that interact with deployed state (e.g., interest accounting, deficit accrual) shipped with extra verification, including legacy-CBOR decode tests.

Open questions ("is this really exploitable?", "do we need to ship this before SNS or can it ride along?") were resolved on the audit day itself, recorded in the remediation plan as locked decisions.

Second pass — remediation (2026-04-23 to 2026-05-01)

Deployment waves

The remediation went out in 13 numbered waves over nine days. Each wave has its own merged PR, deploy timestamp, pre- and post-deploy module hashes, and a 24-hour bake-watch checklist.

Wave	Findings	Theme
1	SP-001, SP-005	Stability pool accounting: removed the latent double-deduction; switched to a single-source-of-truth post-success accounting path; reconciled pool ledger balance to depositor balances on every liquidation
2	AUTH-001, AUTH-002, SP-004, DOS-009	Auth quick wins: stability pool now rejects non-protocol callers on the liquidation-notification boundary; `dev_*` endpoints compile-time gated behind a feature flag and removed from the mainnet wasm and candid surface
3	ICRC-001 through ICRC-004, ICC-006, ICRC-005 backend half	ICRC transfer hygiene: every protocol transfer now generates a stable per-transfer dedup key, sets `created_at_time`, and treats `Duplicate` ledger responses as success; per-ledger fee cache with `BadFee` refresh
4	LIQ-001, ICC-001, ICC-002, ICC-005, ICC-007	Pending-transfer lifecycle: rekeyed `pending_margin_transfers` by `(VaultId, Principal)` so concurrent liquidators can't overwrite each other; compensating refund on the two-phase liquidate-with-reserves path; durable refund queue for the redemption excess path
5	RED-001, LIQ-006, LIQ-007	Oracle staleness and sanity bands: per-collateral price freshness wired into every redemption and liquidation; sanity band on XRC price updates to filter outlier ticks; ReadOnly latch participates in liquidation gating
6	UPG-001, UPG-002, UPG-004, UPG-006	Upgrade safety: post-upgrade decode now has a tiered fallback rather than trapping on first failure; reinstall guards on every canister

Wave	Findings	Theme
		that holds state, refusing to init with non-empty stable memory; raw-offset stable memory layout documented
7	INT-001, INT-003, INT-004	Interest accounting: redemption now updates `accrued_interest`; borrowing-fee multiplier capped to a documented upper bound; `withdraw_partial_collateral` accrues interest before checking the collateral ratio
8 (a–e)	LIQ-002, LIQ-003, LIQ-004, LIQ-005	Liquidation invariants: deterministic liquidation ordering via a sorted-troves index keyed on collateral ratio; `min_vault_debt` enforced on partial-liquidation residuals; ICRC-3 burn proof required before SP-triggered writedowns; bad-debt deficit account that accrues, repays from future fees, and trips the protocol into ReadOnly above a threshold
10	LIQ-008	Mass-liquidation circuit breaker: rolling window over recent liquidations; if cumulative liquidated debt within the window exceeds a configurable ceiling, automatic protocol pause
11	BOT-001	Liquidation bot auto-cancel collateral-return verification: bot can't progress past cancel until it confirms the protocol has received the returned collateral on the ledger
12	BOT-001b (ICC-008)	Same gate, but for the explicit `bot_cancel_liquidation` path (not just the auto-cancel path)
13	BOT-002	Liquidation bot now propagates cancel/return errors on the swap-failure cleanup path so a partial failure can't get silently retried

Verification

After every deploy, the second-pass verification confirmed:

- The remediation actually closed the finding (re-read the post-fix code, walked the previously-exploitable path, confirmed it now traps or returns an error).
- No regression in adjacent behavior (the audit POC tests stayed green for prior waves).
- The deployed module hash matched the merged commit.
- Operational metrics (cycle burn, query latency, vault count, pool balances, redemption volume) stayed inside expected bands during the 24-hour bake window.

By 2026-05-01 the second-pass verification flagged a small remainder: nine findings either needed a separate follow-up wave or had to be deferred-by-design. That set became the input to the third pass.

Third pass — follow-up waves and drift check (2026-05-01 to 2026-05-

02)

Seven follow-up waves

Wave	PR	Findings	What shipped
INT-002 + INT-006	#147	INT-002 (MED), INT-006 (LOW)	Snapshot-then-decrement with saturating restore on `Err` for both interest harvest and treasury drain. Closes a TOCTOU window where concurrent accrual during the inter-canister mint could be silently lost
RED-002 + RED-003	#148	RED-002 (MED), RED-003 (MED)	`validate_mode()` at the top of both redemption endpoints; redemption now refuses in ReadOnly mode. Redemption shortfalls route through the Wave-8e deficit account via a typed `DeficitSource::Redemption(...)` variant rather than being silently socialized via `saturating_sub`
9a	#150	DOS-001, -003, -004, -008	Pagination on every public query that previously returned an unbounded result set. New paged variants alongside the legacy endpoints; legacy endpoints now apply server-side caps; SP `get_pool_events` length argument is clamped
9b	#151	DOS-006, -007	`get_protocol_status` and `get_treasury_stats` cache their per-vault aggregate computation with a 5-second TTL refreshed by the existing 5-minute XRC tick. Live fields (mode, frozen flags, breaker status) bypass the cache and are read fresh from current state
9c	#152	DOS-005	`check_vaults` is sharded to the at-risk CR band using the Wave-8a sorted-troves index. Threshold is the **max** liquidation ratio across all enabled collaterals plus an alert band — vaults in higher-MLR collaterals don't get missed. Periodic full-sweep escape hatch every 12 ticks

Wave	PR	Findings	What shipped
9d	#153	DOS-010, DOS-011	`rumi_analytics` pull cycle is staggered per-source rather than fanning out N concurrent calls every 60 seconds. `add_collateral_token` no longer registers a permanent XRC fetch timer for collaterals that have been Sunset or Deprecated; the gate is a synchronous status check inside the timer closure
ICRC-005 frontend half	#154	ICRC-005 (frontend)	New `ledgerFeeService.ts` queries `icrc1_fee()` live with a 5-minute per-ledger cache and hardcoded fallbacks for offline scenarios. Every former hardcoded fee site (3 services + 5 Oisy executors + 4 Svelte components) migrated

Drift check

The third pass independently re-verified the key second-pass "Resolved-Confirmed" rows against current code:

- **AUTH-001** stability-pool caller gate: still rejecting non-protocol callers, including anonymous.
- **AUTH-002** `dev_*` cfg-gating: every `dev_*` function still wrapped in `#[cfg(feature = "test_endpoints")]` ; mainnet wasm and mainnet candid surface still free of them.
- **LIQ-001** pending-margin two-tuple keying: `pending_margin_transfers` still keyed by `(VaultId, Principal)` everywhere it's read or written.
- **LIQ-002** sorted-troves index maintained on every state-mutation site (open, close, borrow, repay, margin add/remove, all liquidate variants, redemption, fee accrual). Wave 9c's new readers consume keys consistent with the writers.
- **LIQ-005** deficit-account accrual: 6 call sites verified — 5 liquidation paths plus the new RED-002 redemption shortfall path.
- **UPG-006** reinstall guard: present on every audit-time-scope canister. The third pass added the same guard to `rumi_analytics` (which was scaffolded one day after the audit anchor and slipped past the original scope).

Orphan hunt

The third pass also looked for new issues introduced *during* the remediation cycle:

- Every admin endpoint added in a follow-up wave is gated to the developer principal.
- Every new state field has `#[serde(default)]` so legacy CBOR snapshots decode safely.

- No backwards-incompatible candid changes — field additions are appended at the end of records, enum variants are appended at the end of enums (preserving wire encoding).
- No stale `TODO` / `FIXME` comments planted in follow-up commits.

Mainnet hash spot-check

All eight tracked canister hashes match the documented post-deploy hashes from the wave memory files. No undocumented redeploys, no rollbacks, no drift.

Hygiene actions completed at the end of the third pass

Three small loose ends surfaced in the third-pass report and were closed before declaring the cycle complete:

1. **PR #146 merged.** The PocketIC test fences for ICC-002 (refund-on-writedown-failure) and BOT-002 (swap-fail cleanup TransferFailed/ConfirmFailed/SwapFailed branches) had been prepared on a feature branch but not merged. Merged at commit `4f93de5`. The audit POCs are now in main and run on every CI cycle.
2. **`SECURITY.md` added at repo root.** Documents the responsible-disclosure contact, the single-controller posture as deferred-by-design until the SNS migration, and the audit history. Closes the documentation gap the remediation plan called out for the AUTH-003 finding.
3. **`rumi_analytics::init` reinstall guard added.** Mirrors the existing pattern from `liquidation_bot`, `stability_pool`, `rumi_treasury`, `rumi_3pool`, `rumi_amm`, and `rumi_protocol_backend`. Refuses to init with non-empty stable memory so an accidental `--mode reinstall` deploy can't silently wipe historical analytics state. Both items shipped in PR #155, merged at commit `480c5c7`.

Findings status — final tally

Status	Count	Notes
Resolved on mainnet with regression-test fence	60	Includes all 8 HIGH, 30 of 36 MEDIUM, and 22 of 25 LOW. Deployed and bake-watched
Deferred-by-design until SNS migration	5	AUTH-003 single-controller posture (governance-level decision), AUTH-004/-007/-008/-009 admin-rotation hygiene (rides along with the SNS handoff). Documented in `SECURITY.md`
Accepted housekeeping	4	Operational items where the protocol's current scale doesn't justify the change. Explicitly tracked, with watch thresholds, until scale or signals warrant action
Informational	4	Observations recorded but no action required
Total	**73**	

Plus three audit-discovered follow-on findings (BOT-001, BOT-001b/ICC-008, BOT-002) all resolved during Waves 11 to 13 with their own test fences.

What's next

This was a deliberately rigorous internal pass — the kind of review that's usually run before a major step like an SNS launch or a new collateral type. The protocol stack is in the cleanest shape it's been in since v2.

The next routine review is scheduled for early Q3 2026, with two specific triggers that would advance it earlier:

1. **The SNS migration project starting.** SNS handoff is itself an admin-rotation event, and a fresh review at that point will fold in the four governance-hygiene findings currently parked under "deferred to SNS" — confirming on-chain that controller authority, two-phase admin rotation, and proposal-gated config changes all behave as designed.
2. **A new collateral type or significant TVL inflection.** New collaterals introduce XRC plumbing, oracle freshness assumptions, and liquidation-bot integration that should be revalidated against the audit's threat model. Significant TVL inflection (an order-of-magnitude move) is the trigger for revisiting the

deferred housekeeping items (event-log eviction, pre-upgrade serialization layout) where the cost-benefit tradeoff changes with scale.

In the meantime, every deploy continues to run pre-deploy unit and integration tests via the project's hook system; module hashes are recorded on-chain at deploy time and cross-referenced in commit messages; and the protocol's mainnet behavior is continuously instrumented through the analytics canister for the kind of drift that doesn't show up in static review.

Acknowledgments

The audit was structured along the lines of professional third-party engagements seen on similar IC and EVM CDP protocols, with specific attention to the IC's async-message model — the "TOCTOU at every `await`" property that makes IC DeFi auditing different from EVM auditing. Threat models drew from the canister-security skill in the agent toolchain, the IC platform team's published guidance on async safety and stable memory, and the public postmortems of comparable IC DeFi protocols.

Reviewers should feel free to reach out via the responsible-disclosure contact in `SECURITY.md` with questions, follow-up findings, or independent verification work. Findings, fixes, deployed module hashes, and test fences are all traceable through the public commit history of `RumiLabsXYZ/rumi-protocol-v2`.

— Rumi Protocol team, 2026-05-02